

小型开源大语言模型的数学推理能力： 提示方法准确率与回答效率评估

Baizheng Chen

Department of Computer Science

Student ID: 72510800

摘要—本文完成 CS6493 课程项目题目一：大语言模型的数学推理能力评估。我们在 MATH-500、GSM8K 和 AIME 2024 上评估两个小型数学推理模型：Qwen2.5-Math-1.5B 与 DeepSeek-R1-Distill-Qwen-1.5B。方法包括四个基础方法：零样本 Chain-of-Thought、少样本 Chain-of-Thought、Self-Refine 与 Self-Consistency；同时加入两个扩展方法：作为主要纯提示方法的 Plan-and-Solve，以及作为可选工具增强方法的 Tool-Integrated Reasoning。TIR 也可以被解释为轻量 agentic 推理模式，因为模型可以决定何时调用工具、观察结果，并继续完成解题。除准确率和最终回答长度外，本文还报告总生成长度，以及一个基于正确性门控、长度惩罚、答案重复惩罚和过度反思惩罚的行为感知综合分数。

I. 引言

大语言模型能够生成流畅的逐步解答，但数学推理仍然困难，因为模型需要理解符号表达、执行多步逻辑，并输出精确的最终答案。课程项目要求在 MATH-500、GSM8K 和 AIME 2024 上比较不同提示方法，并以准确率和回答长度作为关键指标。

本项目在满足要求的基础上加入了效率视角。一个方法如果需要很长、重复的推理才能得到正确答案，并不等价于一个能简洁得到相同答案的方法。这个区别对 Self-Consistency、Self-Refine、Plan-and-Solve 和工具增强方法尤其重要，因为这些方法可能为同一道题触发多轮生成或工具调用。

本文的主要扩展方法是 `plan_solve`。它是纯 prompt-only 方法，因此可以与 CoT、Self-Refine 和 Self-Consistency 公平比较。本文也报告 `tir`，即 Tool-Integrated Reasoning，作为独立的工具增强扩展。由于 TIR 允许 Python 执行，它不被当作普通 prompt-only baseline；但这种设定也是它的主要研究价值，因为它把

数学推理推进到 reason、tool invocation、observation、continuation 的 agentic 循环。

II. 相关工作

CoT prompting 通过显式中间推理步骤提升模型推理能力 [1]。Self-Consistency 采样多条推理路径并聚合答案，以提升鲁棒性 [2]。Self-Refine 让模型对自身输出进行批判和修订 [3]。Plan-and-Solve 首先要求模型分解任务，然后按计划求解，主要针对零样本推理中的漏步错误 [4]。

工具增强方法将自然语言推理与可执行计算结合。ToRA 表明，通过交替生成推理文本和调用外部计算工具，可以显著提升数学求解能力 [5]。本文的 TIR 不是训练过的 ToRA 模型，而是一个 prompt 层面的工具循环：当模型认为需要计算时生成 Python 代码块，系统执行后再把结果返回给模型。这使 TIR 不只是更长的提示，而是一个小型 agentic workflow：模型判断是否需要工具，生成可执行 action，接收 observation，再修正剩余推理。

数据集和模型也对应已有研究。GSM8K 测试小学数学文字题 [6]，MATH 提供竞赛风格数学题 [7]。Qwen2.5-Math 是面向数学能力优化的 Qwen 模型系列 [8]，DeepSeek-R1-Distill-Qwen 是随 DeepSeek-R1 发布的蒸馏推理模型 [9]。此外，NoWait 研究了“Wait”等显式反思词带来的推理成本 [10]，Dynamic Early Exit 研究了长推理中的答案感知提前停止 [11]。本文只把这些工作作为指标设计动机，而不是声称实现了解码阶段的 early exit。

表 I
提示方法定义。

方法	对话形态	定义
cot_zero	单轮	零样本 Chain-of-Thought。模型只接收目标题目，被要求逐步推理，并以 boxed final answer 结束。
cot_few_shot	单轮	少样本 Chain-of-Thought。提示中加入三个 worked examples，用于约束解题格式和推理风格。
self_refine	多轮	Solve-critique-refine。模型先写初始解答，再批判该解答，最后根据批判重写最终解答。
self_consistency	多采样	采样投票式 CoT。模型独立采样五个单轮解答，抽取最终答案后多数投票，并选择第一次命中投票答案的完整响应作为最终响应。
plan_solve	两轮	Plan-and-Solve。模型先写简短编号计划，再按该计划完成解题。
tir	Agentic 工具循环	Tool-Integrated Reasoning。模型可以生成 Python code block，观察返回的 output block，并基于该证据继续推理。

III. 方法

A. 模型与数据

本文评估 Qwen2.5-Math-1.5B 和 DeepSeek-R1-Distill-Qwen-1.5B。数据集使用项目中的固定采样文件：MATH-500 和 GSM8K 各 50 题，AIME 2024 为 30 题。因此每个模型-方法组合共 130 道题。

B. 提示方法

表 I 定义本文报告的六种方法，并将提示思想和对话形态分开说明。cot_zero 与 cot_few_shot 是单轮方法；self_consistency 不是迭代式多轮对话，而是对同一题发起五次相互独立的单轮采样；self_refine、plan_solve 与 tir 属于多轮或多阶段方法，其中 TIR 还是 agentic 工具循环，因为 Python 执行可以插入模型调用之间，工具输出会成为下一步推理的 observation。

C. 指标

准确率是主指标。当解析出的预测答案与标准答案在数学意义上相等时，该样本记为正确。回答长度被拆成最终回答长度和总生成长度。最终回答长度是提交给 grader 的文本长度；总生成长度包括中间输出，例如 Plan-and-Solve 的 plan + solve、Self-Refine 的 solve + critique + refine、Self-Consistency 的所有采样，以及 TIR 的完整工具 transcript。当存在完整 token 计数元数据时，总生成 token 使用记录到的生成长度，其中包括生成的代码和记录到的 reasoning 文本；否则 token

长度按 $\lceil \max(\text{字符数}/4, \text{单词数}/0.75) \rceil$ 估算。字符长度是原始字符串长度。

每个样本的综合分数先由正确性门控。效率项定义为：

$$\text{efficiency}_i = \text{answer}_i^{0.5} \text{length}_i^{0.3} \text{reflection}_i^{0.2}. \quad (1)$$

最终单样本分数采用 accuracy-first 形式：

$$\text{score}_i = c_i \cdot (0.6 + 0.4 \cdot \text{efficiency}_i), \quad (2)$$

其中 $c_i \in \{0, 1\}$ 表示答案是否正确。因此错题仍为 0，答对样本保留主要准确率贡献，效率项只作为次要修正。三个因子的定义如下。

长度因子。令 L_i 为全流程总生成 token 数， $L_{\text{ref}} = 1024$ ， $\tau_L = 512$ ：

$$\text{length}_i = \exp \left(-\frac{\max(0, L_i - L_{\text{ref}})}{\tau_L} \right). \quad (3)$$

该因子只惩罚过长生成，不惩罚短回答。

答案因子。系统统计答案信号数量 A_i ，包括 \boxed{、 “final answer is”、 “the answer is”、 “answer:” 等。尾部比例 R_i 定义为从第一个答案信号到最终响应末尾的字符比例；若没有答案信号，则 $R_i = 0$ 。答案因子为

$$\text{count}_i = \exp(-\max(0, A_i - 1)), \quad (4)$$

$$\text{tail}_i = \begin{cases} 1, & A_i < 2, \\ \exp \left(-\frac{\max(0, R_i - 0.2)}{0.2} \right), & A_i \geq 2, \end{cases} \quad (5)$$

$$\text{answer}_i = \text{count}_i \cdot \text{tail}_i. \quad (6)$$

它用于惩罚重复声明答案，以及第一次答案之后继续生成过长内容。

反思因子。系统统计反思提示词数量 F_i ，例如 “wait”、“however”、“check”、“verify”、“rethink” 及中文对应词。参考数量为 1， $\tau_F = 2$ ：

$$\text{reflection}_i = \exp \left(-\frac{\max(0, F_i - 1)}{2} \right). \quad (7)$$

本文的 joint score 是样本分数均值。正确样本上的平均 efficiency 作为独立诊断指标报告。

IV. 实验

当同一模型、方法和数据划分存在多条评测记录时，表格只报告 joint score 最高的记录。Overall 行从

表 II
130 道题上的 OVERALL 结果。

模型	方法	Accuracy	Final Tok.	Total Tok.	Joint
DeepSeek-R1-Distill-Qwen-1.5B	<code>cot_zero</code>	0.785	4100.1	4100.1	0.5112
DeepSeek-R1-Distill-Qwen-1.5B	<code>cot_few_shot</code>	0.785	3534.8	3534.8	0.4996
DeepSeek-R1-Distill-Qwen-1.5B	<code>self_refine</code>	0.792	362.3	4709.8	0.5766
DeepSeek-R1-Distill-Qwen-1.5B	<code>self_consistency</code>	0.854	4217.5	21449.8	0.5214
DeepSeek-R1-Distill-Qwen-1.5B	<code>plan_solve</code>	0.892	174.1	2252.7	0.6955
DeepSeek-R1-Distill-Qwen-1.5B	<code>tir</code>	0.908	167.5	1460.3	0.7140
Qwen2.5-Math-1.5B	<code>cot_zero</code>	0.531	2248.2	2248.2	0.5176
Qwen2.5-Math-1.5B	<code>cot_few_shot</code>	0.515	2533.6	2533.6	0.4604
Qwen2.5-Math-1.5B	<code>self_refine</code>	0.508	348.5	1001.1	0.4726
Qwen2.5-Math-1.5B	<code>self_consistency</code>	0.562	848.9	4524.0	0.4789
Qwen2.5-Math-1.5B	<code>plan_solve</code>	0.785	380.9	721.7	0.6570
Qwen2.5-Math-1.5B	<code>tir</code>	0.800	371.9	537.1	0.6681

overall summary 中独立选择，不由单数据集行重新加权计算。

表 II 显示，`plan_solve` 是两个模型上 accuracy 最强的 prompt-only 扩展方法，并且 joint score 高于四个基线。对 Qwen2.5-Math-1.5B，它把 overall accuracy 从 prompt-only baseline 的 0.531–0.562 提升到 0.785。对 DeepSeek-R1-Distill-Qwen-1.5B，更新后的 `plan_solve` 达到 0.892 overall accuracy，并在三个数据划分上都保持较强表现。计入完整生成 token 后，它的总生成成本不再像最终回答长度看起来那样低。

TIR 仍然非常有竞争力，在 DeepSeek-R1-Distill-Qwen-1.5B 上达到最高 overall accuracy 0.908 和 0.7140 joint score；在 Qwen2.5-Math-1.5B 上达到 0.800 accuracy 和 0.6681 joint score。但 TIR 不能与纯提示方法直接等价比较，因为它允许 Python 执行。因此更合理的解释是：TIR 是一种工具增强且带有 agentic 特征的扩展，展示了当模型可以把部分求解过程外化为可执行 action、观察结果并继续推理时，算术和符号计算如何提升准确率；同时它也会因为代码、工具轮次和记录到的 reasoning 带来额外生成成本。

表 IV 说明在 accuracy-first 分数下，效率诊断仍有意义。Self-Consistency 可以有高准确率，但它会采样五个独立输出，因此总生成长度很高。TIR 与 Plan-

and-Solve 的最终回答仍较短，但在计入完整生成长度后总 token 成本更高。这说明最终回答长度和总生成成本必须分开报告。

V. 讨论

最稳妥的最终扩展方法是 Plan-and-Solve。它的解释自然：数学解题通常先确定步骤，再执行计算。它也更公平，因为没有使用外部计算器或 verifier。更新后的实验结果显示，它在两个模型上都带来明显 accuracy 收益，但总生成成本需要和简洁的最终回答分开报告。

TIR 更有亮点，但必须谨慎表述。它不应被写成普通 prompt-only baseline，因为它给模型提供了 Python interpreter。它的收益来自语言推理和工具执行的结合。这个设定应该被突出，而不是被弱化：它是一个紧凑的 agentic pattern，模型先选择 action，接收 observation，再更新最终解答。对于数学推理任务，这尤其有价值，因为小模型常见错误正是算术或符号执行错误。它也指向一个比静态 prompting 更宽的方向：小型数学模型可以和可靠工具结合，形成更强的 solver agent。

行为感知指标的意义在于，单纯 accuracy 会隐藏低效推理。Self-Consistency 通过采样提高准确率，但总生成成本高。DeepSeek 的 CoT 运行也出现非常长的最终回答，即使答对也会降低效率。答案因子和反思因

表 III
各数据集的 ACCURACY。

模型	方法	MATH-500	GSM8K	AIME 2024
DeepSeek-R1-Distill-Qwen-1.5B	cot_zero	0.900	0.900	0.400
DeepSeek-R1-Distill-Qwen-1.5B	cot_few_shot	0.920	0.920	0.333
DeepSeek-R1-Distill-Qwen-1.5B	self_refine	0.880	0.920	0.433
DeepSeek-R1-Distill-Qwen-1.5B	self_consistency	0.920	0.980	0.533
DeepSeek-R1-Distill-Qwen-1.5B	plan_solve	0.940	0.980	0.667
DeepSeek-R1-Distill-Qwen-1.5B	tir	0.960	0.940	0.767
Qwen2.5-Math-1.5B	cot_zero	0.560	0.760	0.100
Qwen2.5-Math-1.5B	cot_few_shot	0.560	0.780	0.000
Qwen2.5-Math-1.5B	self_refine	0.560	0.740	0.033
Qwen2.5-Math-1.5B	self_consistency	0.580	0.840	0.067
Qwen2.5-Math-1.5B	plan_solve	0.860	0.960	0.367
Qwen2.5-Math-1.5B	tir	0.900	0.940	0.400

表 IV
正确样本上的 OVERALL 行为因子均值。

模型	方法	Len.	Ans.	Refl.
DS	cot_zero	0.629	0.248	0.017
DS	cot_few	0.727	0.083	0.039
DS	self_refine	0.286	0.648	0.578
DS	self_cons.	0.010	0.160	0.013
DS	plan_solve	0.608	0.368	0.983
DS	tir	0.726	0.339	0.975
Qwen	cot_zero	1.000	0.918	0.959
Qwen	cot_few	1.000	0.585	0.961
Qwen	self_refine	0.912	0.804	0.914
Qwen	self_cons.	0.515	0.758	0.942
Qwen	plan_solve	0.963	0.368	0.979
Qwen	tir	0.972	0.368	0.955

子进一步帮助诊断重复答案信号和过度自我修正语言。它们不替代 accuracy，而是作为诊断指标使用。

VI. 结论

本文在 MATH-500、GSM8K 与 AIME 2024 上评估了两个小型数学推理模型。最终建议将 Plan-and-Solve 作为主要 proposed prompt-only 方法，因为它公平、简单，且贴合数学解题流程。TIR 可作为可选的工具增强与 agentic 扩展，展示强实验结果，但必须明确说明其 Python 执行设定。最终报告应同时强调 accuracy 与 response efficiency，并区分最终提交答案长度和全流程生成成本。

参考文献

- [1] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems*, 2022.
- [2] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou, “Self-consistency improves chain of thought reasoning in language models,” *arXiv preprint arXiv:2203.11171*, 2022.
- [3] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhunoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, “Self-refine: Iterative refinement with self-feedback,” in *Advances in Neural Information Processing Systems*, 2024.

- [4] L. Wang, W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee, and E.-P. Lim, “Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models,” *arXiv preprint arXiv:2305.04091*, 2023.
- [5] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, M. Huang, N. Duan, and W. Chen, “ToRA: A tool-integrated reasoning agent for mathematical problem solving,” *arXiv preprint arXiv:2309.17452*, 2023.
- [6] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, “Training verifiers to solve math word problems,” *arXiv preprint arXiv:2110.14168*, 2021.
- [7] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt, “Measuring mathematical problem solving with the MATH dataset,” in *Advances in Neural Information Processing Systems*, 2021.
- [8] A. Yang, B. Zhang, B. Hui, B. Gao, B. Yu, C. Li, D. Liu, J. Tu, J. Zhou, J. Lin, K. Lu, M. Xue, R. Lin, T. Liu, X. Ren, Z. Zhang *et al.*, “Qwen2.5-Math technical report: Toward mathematical expert model via self-improvement,” *arXiv preprint arXiv:2409.12122*, 2024.
- [9] DeepSeek-AI, “DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning,” *arXiv preprint arXiv:2501.12948*, 2025.
- [10] C. Wang, Y. Feng, D. Chen, Z. Chu, R. Krishna, and T. Zhou, “Wait, we don’t need to “wait”! removing thinking tokens improves reasoning efficiency,” *arXiv preprint arXiv:2506.08343*, 2025.
- [11] C. Yang, Q. Si, Y. Duan, Z. Zhu, C. Zhu, Q. Li, M. Chen, Z. Lin, and W. Wang, “Dynamic early exit in reasoning models,” *arXiv preprint arXiv:2504.15895*, 2025.